共享 32KiB A 池与本地读写并发：RTL 实施及评估

文档来源 commit：3e8dffdc9dce89cf7f04ff77b2b1cf711dae0039

源文档 SHA-256：5c77999f96f79a5b86f00b8a05e6a2ac554b310916a59755ada10ee2d76aaae8

原文与链接：查看固定版本来源

口径说明：吞吐沿用旧体系的条件估计；RTL功能、局部实测、面积证据与整图条件结果分别报告。

关联 Issue247、PR243；负责人 Melodydt。2026-11-10，Asia/Shanghai。

本轮完成共享 32KiB A 池、本地逻辑读写并发及双 P 接入，实际 SRAM 宏仿真、四配置集成数值测试、默认兼容回归和生产参数模块综合均已通过。两项同时启用时，两 P + 一 U + 共享池共需 **112KiB 物理 SRAM**，较原配置增加 64KiB。

按原评价体系完成的单因素端口敏感性表明：把共享 A 消费宽度从旧假设的 32B/拍限制为实际接口的 16B/拍后，理想本地双端口层的 W8 prefill2 从 **4.398913 降至 3.496459 token/s，-20.5154%。**这些仍是旧 S32/S96 条件模型的结果；当前尚未取得完整原生 RTL 大模型生成的 token/s。

完整数据见结果分析、证据索引、原体系端口敏感性和独立复核。

1. 出发点与实施范围

此前私有 S 扩容研究试图减少 A 的重复搬运，但给两个 P 分别扩容会复制相同的 A 存储。共享池的目标是让两个 P 使用同一份已填好的 A，并直接送入 M 的 A 装载入口，减少 DDR→本地 S→A feeder 中的本地 A 写入和读出。先选择 32KiB，以控制共享容量和外围成本。

本地读写并发解决另一类限制：原四颗 1RW 宏只能在 bank 资源允许时机会性重叠读写，不能保证每拍一读一写。本轮用额外物理存储和活跃位置表提供一个逻辑读端口及一个逻辑写端口，同时保留每 P 16KiB 的逻辑地址空间。

两项能力通过新增双 P 包装层组合。原有 `pu_p_group` 的 `SHARED_A`、`S_TRUE_RW` 默认均为 0；新包装层显式选择参数，不要求旧调用方立即切换。此阶段交付的是可综合模块、原生 P 集成和验证程序，尚未把完整母控、DDR DMA 和大模型编译运行链一起完成。

代码	实施职责
`shared_s/shared_a_pool32.sv`	八宏共享池、填充、对象寿命、双消费者 FIFO 和 128bit A 输出
`shared_s/pu_p_shared_pair.sv`	两 P 与池连接，统一复位，LOAD/RELEASE 排空协调
`pu_leaf_base/pu_p_group.sv`	共享 A 直供选择，本地 B 与共享 A 完成汇合
`s_sram/block_feed_s16k_1r1w.sv`	双物理位置、活跃位置表、有限响应信用、逻辑读写并发
`local_s_p`、LOAD 装载器及存储控制器	后端选择、跳过本地 A、活跃装载范围保护

上表 RTL 路径均位于 `project/rtl/`；测试和复跑入口位于 `project/experiments/shared32_native/`。

2. 共享池的实际数据通路

共享池由 **8×4KiB 的 256×128bit 1RW SRAM 宏**组成，组织为两组、每组四个 lane。一次读同时访问所选组的四个 lane，返回 64B；全局每拍最多批准一次读取，两个 P 用轮询仲裁。同址请求仍分别读取，没有免费广播。

每 P 有独立的 4×512bit FIFO，即 256B，随后以 128bit 容器送入原生 M。单 P 出口上限是 **16B/拍**，两 P 合计最多 32B/拍；内部 64B/拍用于补充有限 FIFO，不能理解为每 P 都有 64B/拍的算术入口。FIFO 信用包括在途读取，直到对应返回行的最后一个有效容器真正握手才释放。背压期间，数据及 bank/row/tile/width 标签保持稳定。

对象最多包含 2048 个 128bit A 容器。每容器对应一行的一个 K8 tile，S16 使用全部 128bit，S8 使用低 64bit，但两者都占 16B。布局为紧凑 tile-major：`word = base128 + tile × rows + row`。

每个 P 同时只处理一条描述符，字段包括 operand bank、rows=1～8、tiles=1～4、A 宽度、11bit 容器基址和 epoch。完整扩展位宽核验 `base + rows×tiles <= object_words`，避免地址加法溢出。错误范围、形状或 epoch 返回错误完成，不产生 A 数据。

对象与 LOAD 的生命周期

阶段	必须满足的条件和行为
BEGIN	仅 EMPTY 可接受；记录 epoch 和 1～2048 个有效容器
FILL	512bit 顺序行写入；mask 按完整 128bit lane 生效；末行只能是连续低位 mask
SEAL	所有有效容器写入完成，且全部写 ACK 已取走，才使对象可读
服务	对象保持只读；每 P 的 done 在最后一个 A 容器真正被接收后产生，并保持到 take
RELEASE	两 P 已接受的 LOAD、描述符完成、FIFO 和在途读取全部排空后，使对象失效

填充侧有两个写响应信用，包含在途写；非法地址或 mask 不写宏、不推进位置。服务期间不能填充。A 已复制到 M 的 operand bank 后，池不再参与其算术，因此 RELEASE 无需额外等待后续 M 计算结束。

一次共享 LOAD 同时发出共享 A 描述符和本地 B 装载。本地装载器跳过 A；若同时复用 B，允许本地侧无数据事务。P 等两侧完成并排空后报告 LOAD done，错误取两侧的 OR。失败 LOAD 可能已经更新 A 或 B 的一部分，**不提供原子回滚**；计算前必须重新完整装载该 operand bank。

包装层先阻止 RELEASE 期间的新 LOAD，再等待内部 P 的装载状态清空，最后把 RELEASE 交给池。这覆盖外部 LOAD 已接受、内部池描述符尚未转发的窗口。`pair_reset = reset || cold_reset` 同时清除池和两 P，防止单边复位留下无法排空的描述符。池复位不清零 SRAM；重新使用须重新 BEGIN、填充并 SEAL。

3. 本地逻辑 1R1W 的做法与代价

每个 P 的四个 lane 各使用两颗真实 1RW 宏，1024bit 活跃位置表记录每个逻辑 128bit word 的最新数据位置。这两份物理空间是可迁移位置，不是始终一致的镜像。

同拍读写时读取活跃位置，将写入送到另一个物理位置并更新映射。同地址采用 read-before-write；读接受时保存位置选择，后续迁移不会改变已接受读的返回数据。普通 128bit 读与 512bit 宽读共享逻辑读端口；普通写与带 lane mask 的宽写共享逻辑写端口。响应队列有限，不能把该能力解释成每个外部客户端都独占带宽。

warm reset 取消请求和响应但保留位置映射及 SRAM；真实宏已经接受的写不回滚。cold reset 清映射并使内容失效，之后必须重写。活跃 LOAD 的本地范围受保护，数据需等待实际写 ACK 才能交付。逻辑容量仍为 16KiB，但物理 SRAM 为 32KiB/P。

4. 容量与 S32 格式边界

此前“单行 A 约 43KiB”指 K=11008、每元素 32bit 的原始数据。旧条件模型直接接受 S32 和 S96；实际 M 的入口是 S8/S16，新增池没有实现 S32 自动转换。

单行 K=11008 的表示	存储量	32KiB 池能否完整存放
旧模型 S32 原值	44032B = 43KiB	否
一个原生 S16 limb	22016B = 21.5KiB	能，但只是三项中的一项
无损 S32 的三个 S16 limb	66048B = 64.5KiB	否

既有条件原生程序用基数 2^15、位移 0/15/30 的三个 S16 limb 表示 S32。当 B 已为原生 S8/S16 时，A 的三个 limb 对应三项乘积，另支付转换、缩放和合并费用；若 B 也为 S32 并拆成三个 limb，通常要处理九项交叉乘积。三个 A limb 同时驻留时，每 K8 组需要 48B；32KiB 最多容纳 682 个完整组，即 5456 个原 S32 位置。若前缀策略要求完整 K32 命令，还要继续向下对齐。

一次只存一个 limb 可以装下全行，但需要分三轮转换、填充、计算和回收，并重新核对 B 重用及 context 寿命；该调度尚未实现。S8 同样占 16B/K8 容器，不能仅按有效数据位宽将池容量翻倍。多行及尾 tile 也必须按真实容器计费。

5. RTL 验证与实际周期

最终 run32 使用 source `a7a36cbb76fa76f6162b666d1f5683cda2a8f7ec`，job102006，node1，76 秒，COMPLETED 0:0；`passed`、`functional_passed` 和模块综合均通过。全部 EDA 在服务器 Slurm 内运行。

- 独立池三个初始化/seed 组合通过：各有 46 次物理读、525 次写、139 个 A 容器握手、15 次预期错误拒绝；两个 FIFO 峰值均为四项。覆盖填充 ACK、背压、同址双 P、64B/16KiB 边界、错误范围/epoch、RELEASE 和复位重填。

- 本地后端三个 ECHO1 组合及一个生产 ECHO0 组合通过；每组连续吞吐测试均在 256 拍完成 256 次宽读和 256 次宽写，256 拍均同拍读写。

- pair 的共享开/关×本地并发开/关四配置，各运行三组初始化，共 12 次。累计 **41472 次主数值比较 + 192 次复位恢复数值比较**通过，覆盖 S8/B8、S16/B8、S16/B16，三行、两个 K 段含尾 tile、八 context、双 P 不同 B、共享前缀与本地尾部、真实 SRAM 写回和读回。

- 生命周期覆盖 RELEASE 与新 LOAD 同拍、LOAD 后一拍 RELEASE、持有完成回执、双 P cold reset 及重填恢复；本地并发配置另测活跃 B 范围写保护与真实同拍宽读写。

下表为三个 seed 一致的微程序周期。“执行段”对应日志 `compute_cycles`，包含测试定义的 LOAD/计算/结果校验等程序工作，不是纯 MAC 延迟。

配置	S8/B8 总周期 / 执行段	S16/B8 前缀 + 尾部 总周期 / 执行段	S16/B16 总周期 / 执行段
共享关、并发关	5053 / 3755	5245 / 3947	7153 / 4703
共享开、并发关	6237 / 3702	6528 / 3921	8388 / 4701
共享关、并发开	3973 / 3755	4165 / 3947	5113 / 4703
共享开、并发开	5277 / 3702	5508 / 3921	6468 / 4701

不能用上表计算整模型加速比。 共享测试为跨 16KiB 边界特意填充 1050 个 word，包含大量不用的前缀；场景 2 的共享配置还执行额外负例，B 事件为 400，对照为 384，工作并不完全相同。总周期还包括初始化、填充、写回 ACK、数值读回和释放。表中差异只能用于核对这个测试程序。

默认兼容由 run29/job102003 独立证明：12 组旧/新默认配置的所有观测握手周期、数据和原回执完全相同，另有三个 cold/warm reset 回归通过。run29 与 run32 的生产 RTL 相同；后续变化为测试程序，兼容结果不借用不同 RTL 的回执。

测试中发现的问题及修正

run26 的池严格 lint 因 based literal 紧邻 `?` 导致语法解析失败；补充分隔空格后，run27 独立单元通过，未放宽新模块的 warning-fatal 检查。设计审查还修复了双 P 与池的统一 cold reset、外部 LOAD/RELEASE 排空窗口，并为旧装载器实例显式接入关闭的 `skip_a`。

run28 的 pair 构建遇到 Verilator 生成的 C++ 未声明 `__Vfork_4__sync`，定位为 cold-reset 数值任务的嵌套 fork 展开。改为两个常驻驱动线程的请求/完成握手，保留双 P 同时开始及全部断言。run31 则在同一 delta 驱动 RELEASE 后立即读取旧 `cfg_ready`，误判已握手；改为下一 posedge 按真实握手边沿检查，43 条 fatal 断言保留。这两次是 TB 修正，未以修改 RTL 或删检查掩盖失败；最终 run32 全部重跑通过，失败回执仍保留。

6. 面积：生产参数和正确对照

两项同时启用时逻辑容量为 80KiB，物理容量为 112KiB。两 P 的双物理位置增加 32KiB，共享池另增加 32KiB。

配置	两 P 物理 SRAM	U	池	总物理 SRAM / 4KiB 宏
原配置	32KiB	16KiB	0	48KiB / 12
仅共享池	32KiB	16KiB	32KiB	80KiB / 20
仅本地并发	64KiB	16KiB	0	80KiB / 20
两项启用	64KiB	16KiB	32KiB	112KiB / 28

run32 用 Yosys、Nangate45 typical 同库综合，SRAM 为真实 Liberty blackbox；本地后端统一生产 `ECHO_WRITE_DATA=0`，主对照是实际 R4 的 bank_rw 后端。

模块	标准单元面积 μm²	SRAM 宏数	比较用途
shared_a_pool32	35924.098	8	新增共享池及外围
block_feed_s16k_1r1w	24787.210	8	新本地并发后端，每 P
block_feed_s16k_bank_rw	6468.854	4	实际 R4 本地后端，每 P
block_feed_s16k_sram	10345.538	4	plain 参考，不作主对照

已覆盖模块的标准单元增量为 $35924.098 + 2\times(24787.210-6468.854)$，即 **0.072560810mm²**。按现有每颗 4KiB 宏 footprint 0.092462050025mm² 复制估算，宏面积从 1.1095446003 增至 2.5889374007mm²，增量 **1.4793928004mm²**。

两项相加为 **1.5519536104mm² 的已覆盖模块增量估计**；未包括 P 外围新增控制、系统互连及布局布线代价，不能称为完整芯片面积增量。没有进行新配置的布局布线、STA、功耗或 300MHz 物理闭合。run27 的 ECHO1/plain 首版综合不再用作最终生产面积对照。

7. 原评价体系：共享 A 端口宽度敏感性

run30/job102004 完成 32 个条件 SystemC 场景：四种原工作负载×两种本地端口层×pool0/32×共享消费者 32/16B。32B 版本的全部 RESULT 与冻结 run25 全字段一致；16B 版本的 pool0 同样全字段一致。两种宽度各完成 725 个独立数值输出比较，并验证实际生成 header 的地址、顺序、FIFO 信用及消费事件宽度。

唯一计时改动是共享 A 消费量子 **32→16B/拍**。容量仍为共享 32KiB，FIFO 为每 P 四项，池全局 64B 服务和两拍附加读延迟不变；本地 A 尾部、B、M、适配器和原聚合重叠合同均保持冻结。下表所有速率单位为条件 token/s。

场景	原本地端口层	R4 无池	池32KiB、32B/拍	池32KiB、16B/拍	16B 相对32B	16B 相对R4
W8 decode512	共享	3.383963	3.449322	3.447389	-0.0561%	+1.8743%
W8 decode512	理想双端口	3.676213	3.700221	3.640199	-1.6221%	-0.9797%
W16 decode512	共享	1.884024	1.932379	1.932369	-0.0006%	+2.5661%
W16 decode512	理想双端口	2.181337	2.192427	2.192413	-0.0006%	+0.5078%
W8 decode2048	共享	2.773324	2.817071	2.815781	-0.0458%	+1.5309%
W8 decode2048	理想双端口	2.978914	2.994659	2.955223	-1.3169%	-0.7953%
W8 prefill2	共享	3.107254	3.894197	3.388659	-12.9818%	+9.0564%
W8 prefill2	理想双端口	3.696697	4.398913	3.496459	-20.5154%	-5.4167%

端口幅宽对 prefill 的影响最大；此前共享32KiB的 4.3989 依赖每 P 32B/拍的共享 A 消费假设，不能作为当前 128bit 接口的现成性能承诺。W8 理想双端口 decode 也出现小幅低于无池对照的结果，说明共享池并非在所有原场景都自动带来净收益。

上述“理想双端口”仍是旧模型的本地端口条件，不是本轮 LVT 后端的整系统校准。模型仍直接接受 S32、K256 和 S96，保留旧 byte-mask 填充桥和重叠调度，没有原生三 limb 转换、K32 命令展开、实际描述符及 BEGIN/SEAL/RELEASE/LOAD 汇合费用。因此这些数值既不是完整原生 RTL token/s，也不是其严格保守下界。prefill2 是两个输入 token 除以整份 trace 时间，不是 decode TPOT。

8. 证据、复跑和剩余工作

run	source commit	作业与实际结果
run27	34b1b520018a73039f1d66396f7a510a39e8adf8	102001；31秒；独立单元及首版综合通过
run29	05d0c73071bb1d8acfaeffaf6b1883b407f56b80	102003；40秒；12组默认兼容＋3组复位通过
run30	05d0c73071bb1d8acfaeffaf6b1883b407f56b80	102004；5分45秒；32个原体系条件场景通过
run32	a7a36cbb76fa76f6162b666d1f5683cda2a8f7ec	102006；76秒；单元、12次pair及生产参数综合通过

均在 node1、4 核 Slurm 分配下结束为 COMPLETED 0:0。工具为 Verilator5.042、g++11.4.0、Yosys0.63+post、SystemC3.0.2；实际工具、库、源文件、输入和输出 SHA 保存在各回执中，不把工具名称当作完整版本锁。

关键 result.json SHA256：

```
run29 e87d24a3861232e76b9ad71d860e22b8706f8748754f59a41cbbae8f3f291e46
run30 a76271fb334081216248ae3f5ad441d56edabdbfaafbdfb28ee4af0ca26162e5
run32 42a6fa657cd1d7febc603bac824050702edd30f1cccd7ff73f5e9c57298e1000
```

服务器为 `yuzelu@10.129.166.172`，运行目录 `/public/home/yuzelu/Melodydt/issue247-area-speed/runNN/`，源文件在 `source/`，日志为 `slurm-<job>.log`，结果在 `results/result.json`。复跑须建立新的 run 目录并按证据索引复制匹配源码、输入及 manifest，不覆盖旧证据；原体系探针还需完整冻结 run25 输入和历史校准回执。

```
# 在已准备好 SHA-bound source、inputs.json 和输入文件的新 run 根目录：
sbatch --parsable --export=ALL,SHARED32_COMPONENTS=all,SHARED32_AREA=1 \
  source/project/experiments/shared32_native/job.sh
sbatch --parsable source/project/experiments/shared32_native/compat_job.sh
sbatch --parsable source/project/experiments/shared32_native/legacy_port_probe/job.sh
```

三项应使用各自的新目录。成功条件分别是原生验证 `passed=true` 且功能/面积通过、兼容 12 + 3 回执通过、条件探针 32 场景和完整对照通过。所有 EDA 只能在服务器 Slurm 运行；本地 JSON 分析及管理检查不替代这些验证。

实际部署还需：DDR 到有限拼接/转换缓存及顺序 pool fill 的 DMA 桥；编译器与母控的 native tile-major 布局、epoch、前缀/尾部和 LOAD/算术依赖调度；S32 三 limb 转换与正确收费的累计方案；真实权重/KV、完整图和 token 采样数值验证；系统顶层接入及物理闭合。当前池按完整 128bit lane 写入，旧模型任意 byte 交集写入不能当作 DMA 桥已经完成。

PR243 仍待独立审查，未合并，不把 Issue 标为 done。整理本报告时，管理 CI 显示 failure 且无可见 steps，原因尚未确定；保留失败状态并继续核查，不绕过。此前 run24/25 旧证据 bundle 上传被拒且未重传，本阶段新授权的源码、报告和证据单独交付，不把旧 bundle 描述为已在线完成。登记与看板是否同步以任务实际回执为准。